

# DEEP LEARNING – HANDWRITTEN EXAM NOTES

## UNIT 1: Linear Algebra, Biological & Artificial Neurons, Perceptron

B.Tech Mid-1 Examination Preparation | Complete Question Bank Solutions

### ----- PART A – SHORT ANSWER QUESTIONS -----

#### Question 1

Define singular matrix

#### Definition

A square matrix  $A$  is called a **Singular Matrix** if its determinant is equal to zero, that is:  
 $\det(A) = |A| = 0$ .

#### Explanation

When the determinant of a matrix is zero, the matrix does not possess a multiplicative inverse (it is **non-invertible**). Geometrically, a singular transformation squashes  $n$ -dimensional space into a lower dimension, losing data. In Deep Learning, singular matrices cannot be inverted during optimization algorithms (such as Second-Order Newton-Raphson methods).

#### Important Points

- **Determinant:**  $|A| = 0$ .
- **Inverse:** The inverse matrix  $A^{-1}$  **does not exist** because  $A^{-1} = \text{adj}(A) / |A|$  (division by zero is undefined).
- **Linear Dependence:** The rows or columns of a singular matrix are linearly dependent.
- **Eigenvalues:** A singular matrix always has at least one eigenvalue equal to 0.
- **Matrix Rank:** The rank of the matrix is strictly less than its dimension ( $\text{Rank}(A) < n$ ).

#### Example

$$\text{Let } A = \begin{pmatrix} 2 & 4 \\ 1 & 2 \end{pmatrix}$$

$$|A| = (2 \times 2) - (4 \times 1) = 4 - 4 = 0$$

Since  $|A| = 0$ , matrix  $A$  is a **Singular Matrix**.

## Question 2

Define symmetric and skew symmetric matrices

### 1. Symmetric Matrix

**Definition:** A square matrix  $A$  is called a **Symmetric Matrix** if it is equal to its transpose, that is:  $A^T = A$  (or  $a_{ij} = a_{ji}$  for all  $i, j$ ).

- Elements placed symmetrically across the main diagonal are equal.
- Covariance matrices in PCA and Hessian matrices in optimization are always symmetric.

$$\text{Example: } A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 5 & 6 \\ 3 & 6 & 9 \end{bmatrix} \Rightarrow A^T = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 5 & 6 \\ 3 & 6 & 9 \end{bmatrix} = A \text{ (Symmetric)}$$

### 2. Skew-Symmetric (Antisymmetric) Matrix

**Definition:** A square matrix  $A$  is called a **Skew-Symmetric Matrix** if its transpose is equal to its negative, that is:  $A^T = -A$  (or  $a_{ij} = -a_{ji}$  for all  $i, j$ ).

- For diagonal entries (where  $i = j$ ):  $a_{ii} = -a_{ii} \Rightarrow 2a_{ii} = 0 \Rightarrow a_{ii} = 0$ . Thus, all principal diagonal elements must be zero.

$$\text{Example: } B = \begin{bmatrix} 0 & 2 & -3 \\ -2 & 0 & 4 \\ 3 & -4 & 0 \end{bmatrix} \Rightarrow B^T = \begin{bmatrix} 0 & -2 & 3 \\ 2 & 0 & -4 \\ -3 & 4 & 0 \end{bmatrix} = -B \text{ (Skew-Symmetric)}$$

## Question 3

Explain one hot encoding with example

### Definition

**One-Hot Encoding** is a categorical preprocessing method where discrete categorical labels are converted into binary vectors containing all 0s except for a single 1 at the index representing that specific category.

### Explanation

Neural networks are mathematical function approximators. If we encode categories like "Red", "Green", "Blue" as integers 1, 2, 3, the model would assume that Blue (3) > Green

(2) > Red (1). One-hot encoding removes this false numeric ordering, placing each category in its own orthogonal dimension with equal pairwise distance.

### Example

Consider a dataset with 3 distinct categories: {Red, Green, Blue}:

| Category Label | Integer Encoding (Wrong for DL) | One-Hot Encoded Vector [Red, Green, Blue] |
|----------------|---------------------------------|-------------------------------------------|
| Red            | 1                               | [1, 0, 0]                                 |
| Green          | 2                               | [0, 1, 0]                                 |
| Blue           | 3                               | [0, 0, 1]                                 |

### Important Points

- Vector dimension equals total number of classes (N).
- Euclidean distance between any two one-hot vectors is always  $\sqrt{2}$  (fair and unbiased).
- Standard target representation for Multi-Class classification using **Softmax Activation** and **Categorical Cross-Entropy Loss**.

### **Question 4**

**List out the components of Biological neuron**

### List of Components

1. **Dendrites:** Tree-like fibers that act as receiving antennas, collecting electrical and chemical signals from adjacent neurons.
2. **Soma (Cell Body):** The central spherical body containing the nucleus. It integrates (sums) incoming excitatory and inhibitory potentials.
3. **Axon Hillock:** The conical junction connecting soma and axon; triggers the action potential if threshold is reached.
4. **Axon:** A long transmission tube carrying electrical nerve impulses (action potentials) away from the cell body.
5. **Myelin Sheath & Nodes of Ranvier:** Insulating fatty layers along the axon that enable rapid saltatory impulse propagation.
6. **Axon Terminals & Synapse:** Terminal branches that release chemical neurotransmitters across the synaptic cleft (microscopic gap) to communicate with subsequent neurons.

### Diagram

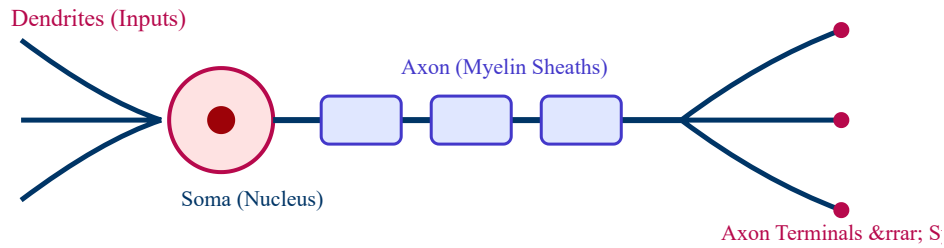


Figure 1.1: Schematic diagram of a Biological Neuron

### Question 5

What are the components in a computational unit?

#### Definition

A **computational unit** (Artificial Neuron / Perceptron Node) is the fundamental processing node in an artificial neural network.

#### Key Components

1. **Input Features** ( $x_1, x_2, \dots, x_n$ ): Numeric signals received from data.
2. **Synaptic Weights** ( $w_1, w_2, \dots, w_n$ ): Strength parameters that scale each incoming input.
3. **Bias** ( $b$ ): An offset parameter allowing the model to shift the activation curve.
4. **Net Summation Unit** ( $\Sigma$ ): Computes the linear dot product:  $z = \sum w_i x_i + b$ .
5. **Activation Function** ( $f$ ): Applies a non-linear mapping:  $y = f(z)$ .
6. **Output** ( $y$ ): The computed output value transmitted downstream.

### Question 6

Difference between linearly separable and non-linearly separable datasets

| Aspect                           | Linearly Separable Dataset                                                                | Non-Linearly Separable Dataset                                                              |
|----------------------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Definition                       | Data classes can be completely separated by a single straight line, plane, or hyperplane. | Data classes cannot be partitioned by a single flat hyperplane; requires non-linear curves. |
| Decision Boundary                | Flat linear boundary: $w^T x + b = 0$ .                                                   | Curved, circular, or multi-segmented boundary.                                              |
| Solvability by Single Perceptron | Solvable with 100% classification accuracy.                                               | Cannot be solved by a single-layer perceptron.                                              |
| Examples                         | AND Gate, OR Gate, linearly distinct clusters.                                            | XOR Gate, XNOR Gate, concentric circles, spiral datasets.                                   |

Required  
Architecture

Single Perceptron unit.

Multi-Layer Perceptron (MLP) with  
non-linear activation functions.

### Question 7

What is the significance of the Perceptron Convergence Theorem?

#### Significance (8 Detailed Points)

1. **Mathematical Learning Guarantee:** It provides a rigorous mathematical proof that if a training dataset is linearly separable, the Perceptron Learning Algorithm will unconditionally find a hyper-plane that separates the classes with zero training error.
2. **Finite Step Convergence:** It proves that convergence is achieved in a finite number of weight updates ( $k$ ), bounded by  $k \leq (R / \gamma)^2$ , where  $R$  is the maximum input radius and  $\gamma$  is the separation margin.
3. **Weight Initialization Independence:** Convergence is guaranteed regardless of whether the initial weight vector is zero or set to arbitrary random numbers.
4. **Error-Driven Optimization:** The theorem mathematically validated error-driven learning: weights change only upon misclassification and stay unchanged upon correct prediction.
5. **Theoretical Milestone of AI:** Published by Frank Rosenblatt and proved by Novikoff (1962), it was the first proof that machines can iteratively learn classification rules directly from data.
6. **Clear Boundary of Limitations:** It explicitly establishes that if data is non-linearly separable ( $\gamma = 0$ , e.g. XOR), the algorithm will never terminate, formally highlighting the boundary of single-layer units.
7. **Foundation of Support Vector Machines (SVM):** The bound depends inversely on the margin ( $\gamma^2$ ). This insight motivated the development of Maximum Margin Classifiers and SVMs to maximize this margin.
8. **Conclusion:** The theorem is the foundation stone of supervised linear learning, confirming that simple local updates can yield guaranteed global linear solutions.

#### ----- PART B – ESSAY / 10-MARK QUESTIONS -----

### Essay Question 1

Explain various types of matrices with examples

#### 1. Introduction

A matrix is an ordered rectangular array of numbers arranged into rows and columns. In Deep Learning, matrices represent dataset batches, weights, activations, and linear transformations.

## 2. Types of Matrices with Definitions & Examples

### 1. Row Matrix:

A matrix containing only one row of order  $1 \times n$ .

Example:  $A = \begin{bmatrix} 2 & 7 & -3 \end{bmatrix}$  (Order  $1 \times 3$ ) — Represents an input feature vector.

### 2. Column Matrix:

A matrix containing only one column of order  $m \times 1$ .

Example:  $B = \begin{bmatrix} 4 \\ 6 \\ 1 \end{bmatrix}$  (Order  $3 \times 1$ ) — Represents neuron bias and output vectors.

### 3. Square Matrix:

A matrix where number of rows equals number of columns ( $m = n$ ).

Example:  $C = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$  (Order  $2 \times 2$ ) — Used in Covariance & Hessian matrices.

### 4. Diagonal Matrix:

A square matrix where all off-diagonal elements are zero ( $a_{ij} = 0$  for  $i \neq j$ ).

Example:  $D = \begin{bmatrix} 5 & 0 & 0 \\ 0 & 9 & 0 \\ 0 & 0 & 2 \end{bmatrix}$  — Used in Singular Value Decomposition (SVD).

### 5. Scalar Matrix:

A diagonal matrix whose principal diagonal elements are all equal.

Example:  $S = \begin{bmatrix} 6 & 0 \\ 0 & 6 \end{bmatrix}$

### 6. Identity (Unit) Matrix (I):

A diagonal matrix where all main diagonal entries are 1.  $A \cdot I = I \cdot A = A$ .

Example:  $I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$  — Used in Ridge Regularization ( $X^T X + \lambda I$ ).

### 7. Zero / Null Matrix (O):

A matrix where all entries are zero. Used for zero-initializing weight and gradient accumulators.

Example:  $O = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$

### 8. Upper & Lower Triangular Matrices

Upper:  $a_{ij} = 0$  for  $i > j$ . Lower:  $a_{ij} = 0$  for  $i < j$ .

Upper:  $U = \begin{bmatrix} 2 & 3 \\ 0 & 5 \end{bmatrix}$  Lower:  $L = \begin{bmatrix} 4 & 0 \\ 1 & 7 \end{bmatrix}$  (Used in LU Decomposition)

### 9. Symmetric Matrix:

A square matrix where  $A^T = A$ . PCA covariance matrices are always symmetric.

Example:  $A = \begin{bmatrix} 3 & 5 \\ 5 & 8 \end{bmatrix} \Rightarrow A^T = A$

### 10. Skew-Symmetric Matrix:

A square matrix where  $A^T = -A$ . Diagonal elements are always zero.

Example:  $B = \begin{bmatrix} 0 & 4 \\ -4 & 0 \end{bmatrix} \Rightarrow B^T = -B$

### 11. Orthogonal Matrix:

A square matrix where transpose equals inverse:  $A^T A = A A^T = I \Rightarrow A^{-1} = A^T$ . Preserves vector lengths.

### 12. Singular & Non-Singular Matrices:

- Singular:  $|A| = 0$  (Non-invertible).
- Non-Singular:  $|A| \neq 0$  (Invertible).

**Conclusion:** Matrices are the foundational data structures in Deep Learning that power vectorized forward/backward passes and dimensional transformations.

## Essay Question 2

**Explain the steps involved in PCA algorithm**

### 1. Introduction

**Principal Component Analysis (PCA)** is an unsupervised linear dimensionality reduction algorithm. It transforms high-dimensional data ( $d$  dimensions) into a smaller number of uncorrelated dimensions ( $k$  components,  $k < d$ ) while preserving the maximum variance.

### 2. Complete Step-by-Step Procedure

#### Step 1: Standardize / Center the Dataset

Compute the mean of each feature and subtract it so each feature has zero mean:

$$\mu_j = (1/n) \sum x_{ij} \Rightarrow X_{\text{centered}} = X - \mu$$

#### Step 2: Compute the Covariance Matrix ( $\Sigma$ )

Calculate the  $d \times d$  symmetric covariance matrix capturing feature variances and correlations:

$$\Sigma = (1/n) \cdot X_{\text{centered}}^T \cdot X_{\text{centered}}$$

#### Step 3: Calculate Eigenvalues and Eigenvectors

Solve the characteristic equation of the covariance matrix:

$$\det(\Sigma - \lambda I) = 0 \Rightarrow \text{Eigenvalues } (\lambda_1, \lambda_2, \dots, \lambda_d)$$

$$(\Sigma - \lambda_i I) v_i = 0 \Rightarrow \text{Eigenvectors } (v_i)$$

- **Eigenvectors ( $v$ ):** Directions of principal axes of highest variance.
- **Eigenvalues ( $\lambda$ ):** Magnitude of variance explained along each axis.

#### Step 4: Sort Eigenpairs in Descending Order

Arrange eigenvalues from highest to lowest:  $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$ . The vector with  $\lambda_1$  is the 1st Principal Component.



### Step 5: Form the Transformation Projection Matrix (W)

Select the top  $k$  eigenvectors corresponding to the  $k$  largest eigenvalues to build matrix  $W$  of size  $d \times k$ :

$$W = [v_1, v_2, \dots, v_k] \quad (\text{Size: } d \times k)$$

### Step 6: Project Original Data onto New Subspace

Multiply the centered dataset by  $W$  to obtain the reduced  $n \times k$  dataset  $Y$ :

$$Y = X_{\text{centered}} \cdot W \quad (\text{Size: } n \times k, \text{ where } k < d)$$

### 3. Flowchart Diagram of PCA

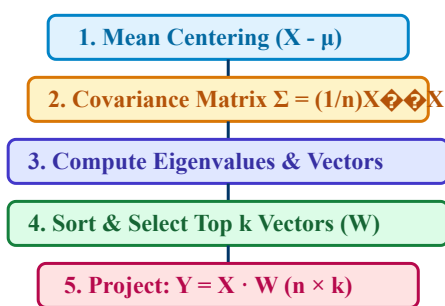


Figure 1.3: Flow of Principal Component Analysis (PCA)

**Conclusion:** PCA compresses features, speeds up neural network training, removes multicollinearity, and enables 2D/3D data visualization.

### Essay Question 3

**Explain the structure of a biological neuron with a neat diagram**

#### 1. Introduction

A **Biological Neuron** is an electrically excitable nerve cell in the human brain that receives, processes, and transmits information via electrochemical signals. It forms the biological inspiration for artificial neural networks.

#### 2. Neat Diagram of Biological Neuron

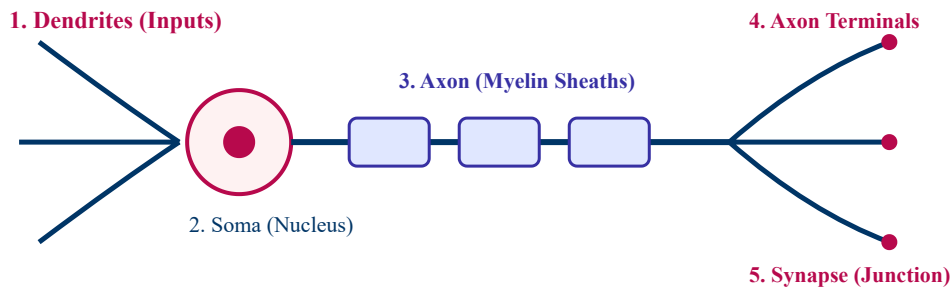


Figure 1.4: Complete Labeled Structure of Biological Neuron

### 3. Detailed Component Explanations

- 1. Dendrites:** Branching nerve extensions that receive electrochemical incoming signals from other neurons.
- 2. Soma (Cell Body):** Integrates and sums all excitatory and inhibitory input potentials.
- 3. Axon Hillock:** The trigger zone that fires an **Action Potential** if the accumulated potential exceeds the membrane threshold ( $-55$  mV).
- 4. Axon & Myelin Sheath:** Axon conducts the action potential. Myelin sheaths insulate the axon to speed up conduction up to  $100$  m/s via Nodes of Ranvier.
- 5. Axon Terminals & Synapse:** Terminals release chemical neurotransmitters across the synaptic cleft to stimulate downstream neurons.

### 4. Biological vs Artificial Neuron Comparison

| Biological Neuron | Artificial Neuron                    | Function                     |
|-------------------|--------------------------------------|------------------------------|
| Dendrites         | Inputs ( $x_i$ ) & Weights ( $w_i$ ) | Receive and scale signals    |
| Soma              | Summation Unit ( $\Sigma + b$ )      | Integrates weighted inputs   |
| Action Threshold  | Activation Function $f(z)$           | Threshold firing / squashing |
| Axon              | Output Signal ( $y$ )                | Transmits final output       |
| Synapse           | Weights ( $w_i$ )                    | Strength of connection       |

#### Essay Question 4

What is a perceptron? Explain the steps involved in training a perceptron in deep learning

### 1. Introduction & Mathematical Model

A **Perceptron** is a single-layer artificial neuron developed by **Frank Rosenblatt** in **1958** for binary classification of linearly separable patterns.

**Net Sum:**  $z = \sum_{j=1}^n w_j x_j + b = \mathbf{w}^T \mathbf{x} + b$

**Activation:**  $\hat{y} = 1$  if  $z \geq 0$ , else  $\hat{y} = 0$  (Step Function)

## 2. Step-by-Step Training Algorithm

### 1. Step 1: Initialization

Initialize weights  $\mathbf{w} = [w_1, \dots, w_n]$  and bias  $b$  to 0 or small random values. Choose a learning rate  $\eta \in (0, 1]$  (e.g.  $\eta = 0.1$ ).

### 2. Step 2: Apply Training Sample

For each training pair  $(x^{(i)}, y^{(i)})$  where  $y^{(i)} \in \{0, 1\}$ :

### 3. Step 3: Calculate Weighted Sum

$$z = \sum w_j x_{j(i)} + b$$

### 4. Step 4: Compute Predicted Output ( $\hat{y}$ )

$$\hat{y} = 1 \text{ if } z \geq 0 \text{ else } 0$$

### 5. Step 5: Calculate Error ( $e$ )

$$e = y^{(i)} - \hat{y}$$

### 6. Step 6: Update Weights and Bias

$$w_j \leftarrow w_j + \eta \cdot (y^{(i)} - \hat{y}) \cdot x_j^{(i)}$$

$$b \leftarrow b + \eta \cdot (y^{(i)} - \hat{y})$$

### 7. Step 7: Epoch Iteration

Repeat Steps 2 to 6 over multiple passes (epochs) until zero classification errors remain across the entire dataset.

## 3. Example: 1 Epoch on AND Gate with $\eta = 0.1$ , $w_1=0$ , $w_2=0$ , $b=0$

| $x_1$ | $x_2$ | Target ( $y$ ) | $z = w_1 x_1 + w_2 x_2 + b$ | Pred ( $\hat{y}$ )    | Error ( $y - \hat{y}$ ) | Weight Updates            |
|-------|-------|----------------|-----------------------------|-----------------------|-------------------------|---------------------------|
| 0     | 0     | 0              | 0                           | 1 (since $0 \geq 0$ ) | -1                      | $b = 0 + 0.1(-1) = -0.1$  |
| 0     | 1     | 0              | -0.1                        | 0                     | 0                       | No change                 |
| 1     | 0     | 0              | -0.1                        | 0                     | 0                       | No change                 |
| 1     | 1     | 1              | -0.1                        | 0                     | +1                      | $w_1=0.1, w_2=0.1, b=0.0$ |

**Conclusion:** Perceptron training iteratively shifts the decision boundary until all training points are correctly classified.

## Essay Question 5

State and explain Perceptron Convergence Theorem; explain why perceptron cannot solve XOR

### PART 1: Perceptron Convergence Theorem

**Statement:** "If a training dataset  $D$  is linearly separable by a geometric margin  $\gamma > 0$ , the Perceptron Learning Algorithm is guaranteed to converge to a separating hyperplane with zero errors within a finite number of weight updates ( $k$ )."

**Upper Bound Formula:**  $k \leq (R / \gamma)^2$

Where:  $R = \max \|x_i\|$  (Maximum radius of input vectors),  $\gamma$  = Separation margin.

### PART 2: Mathematical Proof Why Perceptron Cannot Solve XOR

The XOR truth table requires satisfying four simultaneous linear conditions:

| $x_1$ | $x_2$ | Target $y$ | Condition: $(w_1x_1 + w_2x_2 + b)$ | Simplified Result       |
|-------|-------|------------|------------------------------------|-------------------------|
| 0     | 0     | 0          | $w_1(0) + w_2(0) + b < 0$          | (1) $b < 0$             |
| 0     | 1     | 1          | $w_1(0) + w_2(1) + b \geq 0$       | (2) $w_2 + b \geq 0$    |
| 1     | 0     | 1          | $w_1(1) + w_2(0) + b \geq 0$       | (3) $w_1 + b \geq 0$    |
| 1     | 1     | 0          | $w_1(1) + w_2(1) + b < 0$          | (4) $w_1 + w_2 + b < 0$ |

#### Proof of Contradiction:

- Adding inequalities (2) and (3):  $(w_2 + b) + (w_1 + b) \geq 0 \Rightarrow w_1 + w_2 + 2b \geq 0$ .
- Rewrite as:  $(w_1 + w_2 + b) + b \geq 0$ .
- From (4),  $(w_1 + w_2 + b) < 0$  (negative). From (1),  $b < 0$  (negative).
- Adding two strictly negative numbers must yield a strictly negative number ( $< 0$ ), directly contradicting  $\geq 0$ .
- Conclusion:** No line exists to separate XOR. A Multi-Layer Perceptron (MLP) with non-linear hidden layers is strictly required.

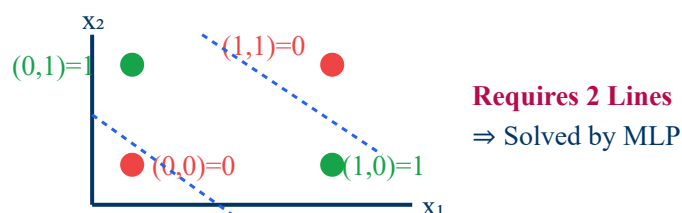


Figure 1.5: XOR Non-Linearity Problem

Quick Formula Reference

| Term              | Formula                                        |
|-------------------|------------------------------------------------|
| Singular Matrix   | $ A  = 0 \Rightarrow A^{-1} \text{ undefined}$ |
| Symmetric Matrix  | $A^T = A$                                      |
| Skew-Symmetric    | $A^T = -A \quad (\text{diagonal} = 0)$         |
| PCA Covariance    | $\Sigma = (1/n) X^T X$                         |
| Perceptron Output | $y = f(w^T x + b)$                             |
| Perceptron Update | $w \leftarrow w + \eta(y - \hat{y})x$          |
| Convergence Bound | $k \leq (R / \gamma)^2$                        |